
Architecture & the Backend

01

Recap



02

Systems Thinking



03

SWE Principles



04

More Design Patterns



05

Architecture Patterns



Agenda

CRITICAL RENDERING PATH

1. Get the initial HTML 
2. Parse the HTML into the DOM
3. Parse CSS into the CSSOM
4. Create render tree
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)

4 COMMONALITIES

1. Components
2. State Handling
3. Special rendering (ex: VDOM)
4. **Dependency Injection**

WHY DESIGN PATTERNS?

01

Proven solutions

These are approaches that have been well tested by the developer community

03

Provides faster communication

You now have the same language to describe a problem and its subsequent solution

02

Reusable

Patterns are robust and can be easily adapted to fit needs

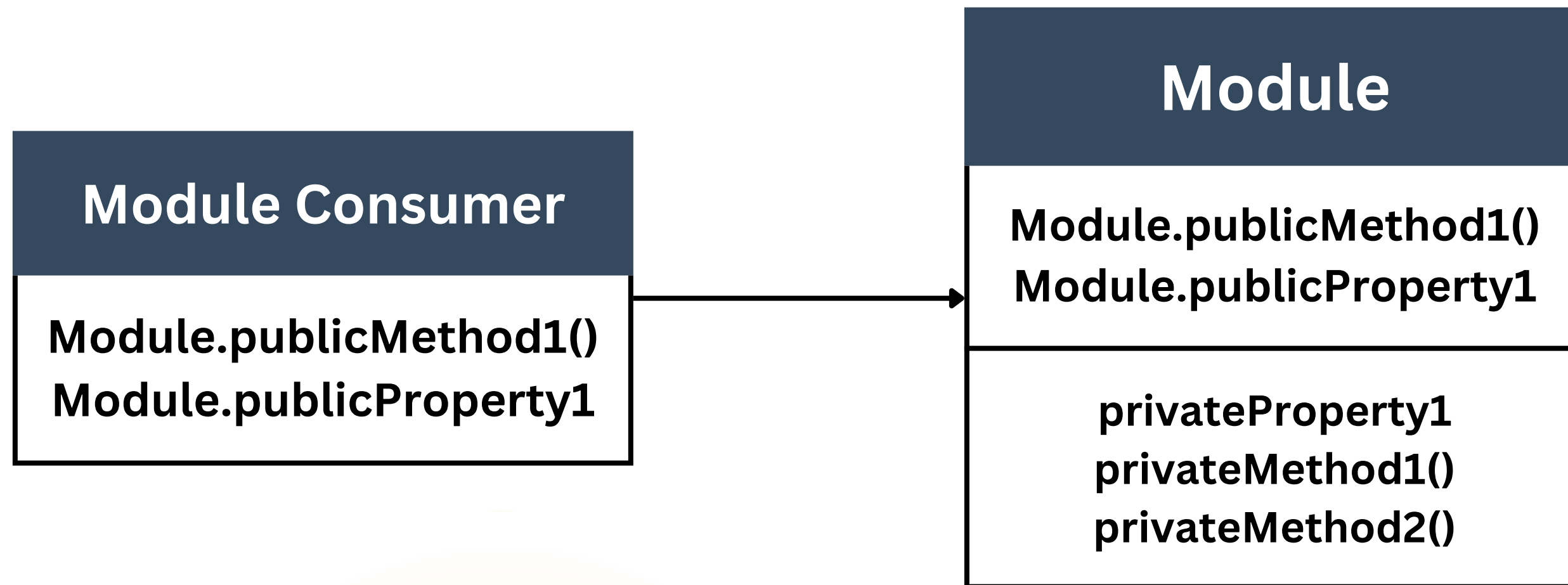
04

Makes code less spaghetti like

Software systems can have better structure

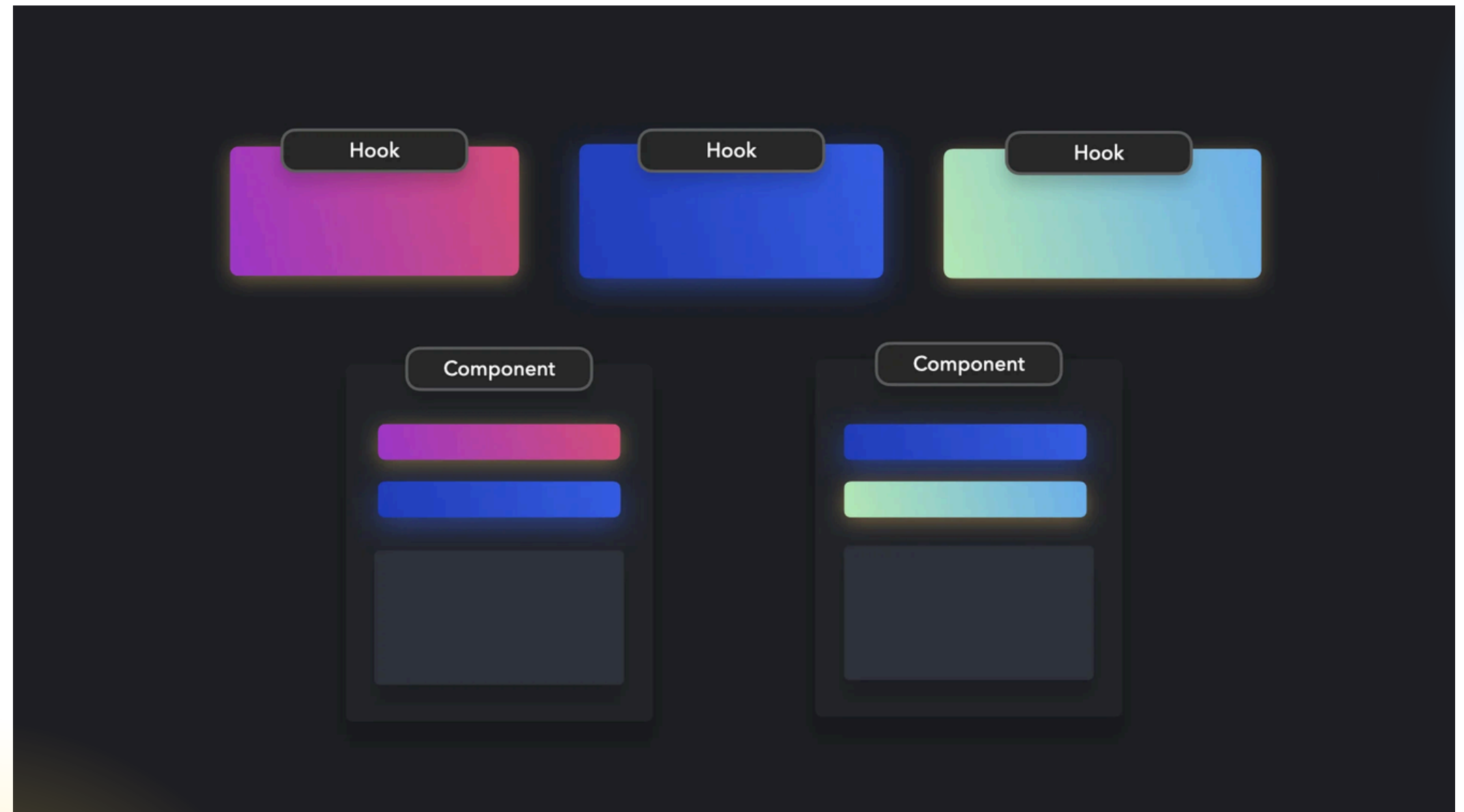
THE MODULE PATTERN

- Provides private/public encapsulation
- Allows us to use import/export
- Prevents our functions/variables from polluting the global namespace
- Closure -- things defined in the module stay in the module unless you define otherwise



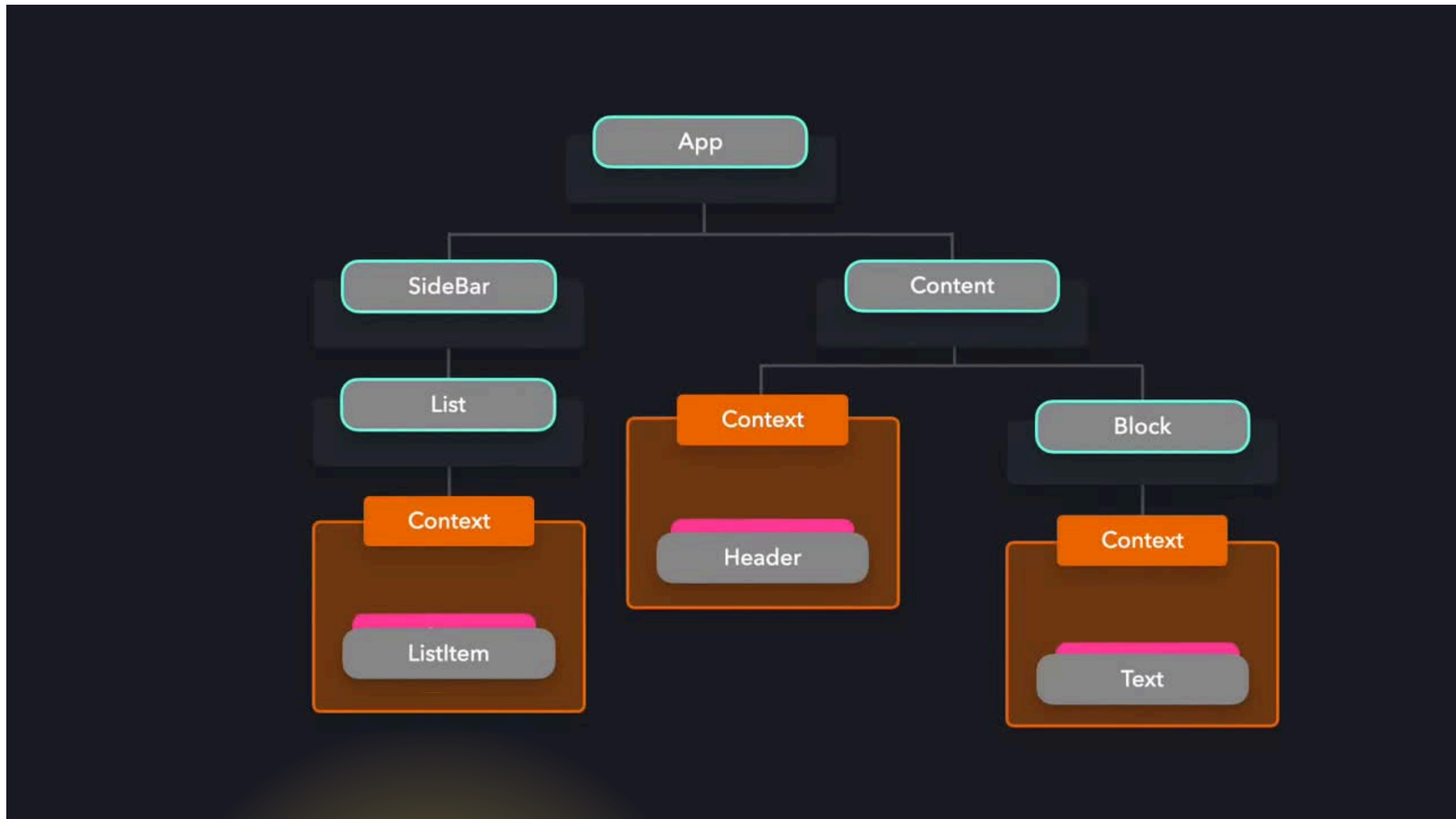
THE HOOKS PATTERN

- React specific design pattern that allows us to dictate what is known as a **reactive effect** (side effect at different points of the component lifecycle)
 - We've seen a few of these already — useState (for instance)
- Other hooks:
 - useEffect
 - useLayoutEffect
 - useMemo
 - useContext
 - useReducer
 -many more
 - And custom hooks



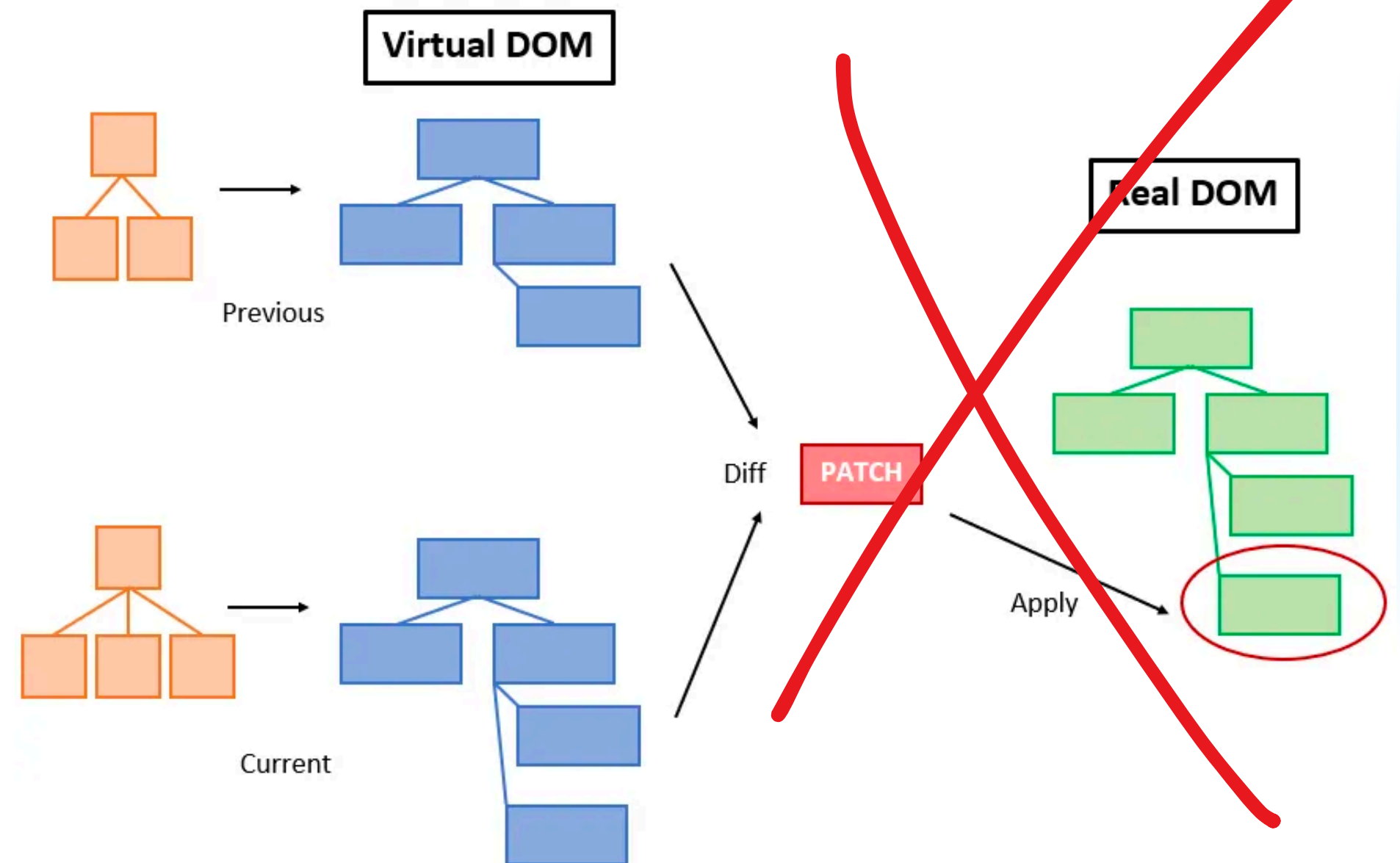
DEPENDENCY INJECTION (DI)

DI – only passing in the data to the components that need the data (context)



VIRTUAL DOM (VDOM)

1. Construct initial virtual DOM
2. When changes are made:
 - a. Rerender a new virtual DOM tree
 - b. Diff against previous virtual DOM tree
3. Make changes to the actual (physical) DOM

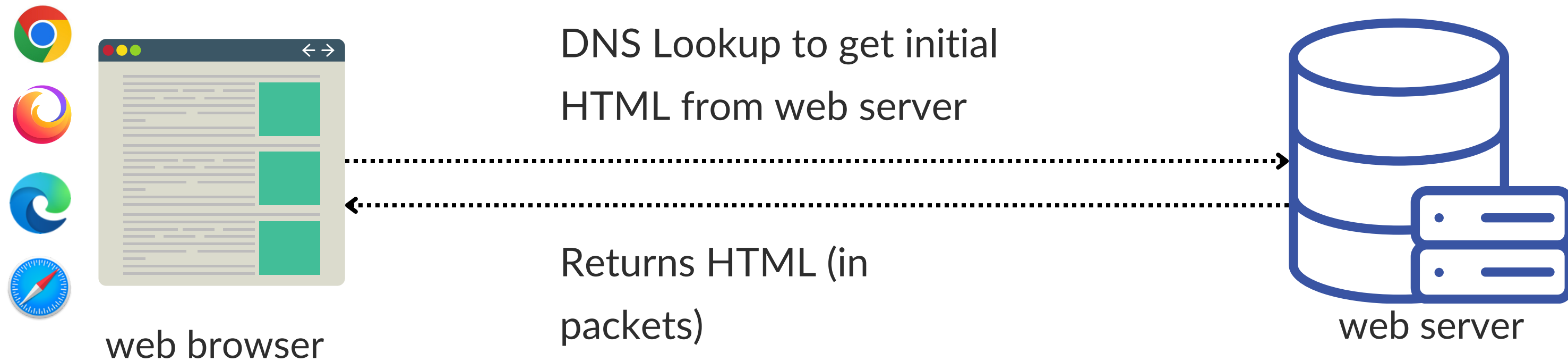


CRITICAL RENDERING PATH

1. Get the initial HTML 
2. Parse the HTML into the DOM
3. Parse CSS into the CSSOM
4. Create render tree
5. Determine where render tree nodes should appear on the page (layout)
6. Paint nodes onto screen (rasterize)

HOW BROWSERS WORK

1. Get the initial HTML



CSR

Client Side Rendering

Pros:

- **(Speedy navigation)** - No need to refresh when navigating to another page

Cons:

- **Giant bundle sizes** - especially as apps become more complex
- **Poor SEO**



SSR

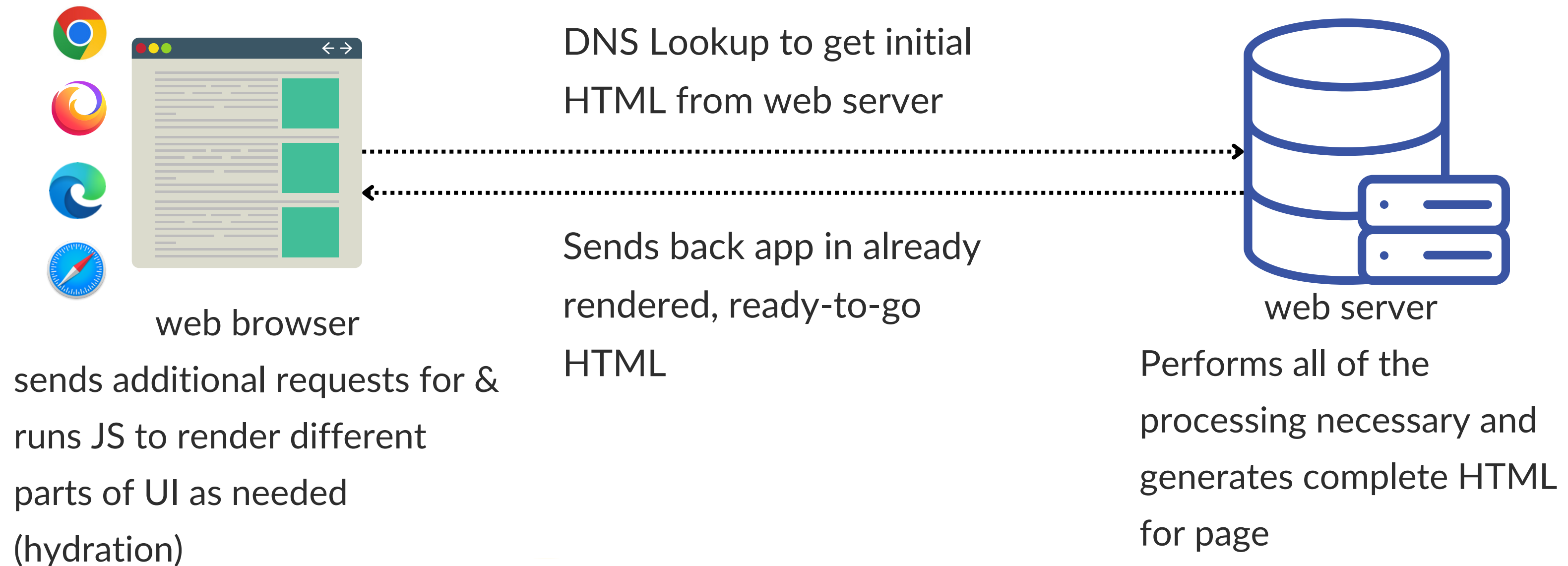
Server Side Rendering

Pros:

- Minimal JS is sent to client
- Reduced roundtrips for data fetching
- Better SEO

Cons:

- Higher server compute costs



NEXT.JS

- Production grade React framework
 - Routing
 - SSR
 - CSR → ‘use client’
 - ex: when using some hooks
- App router



Recap | the FE

In your learning doc:

- When would you use CSR as opposed to SSR and vice versa?
- When would you implement dependency injection in the FE?
- What are you still curious about?

Systems Thinking

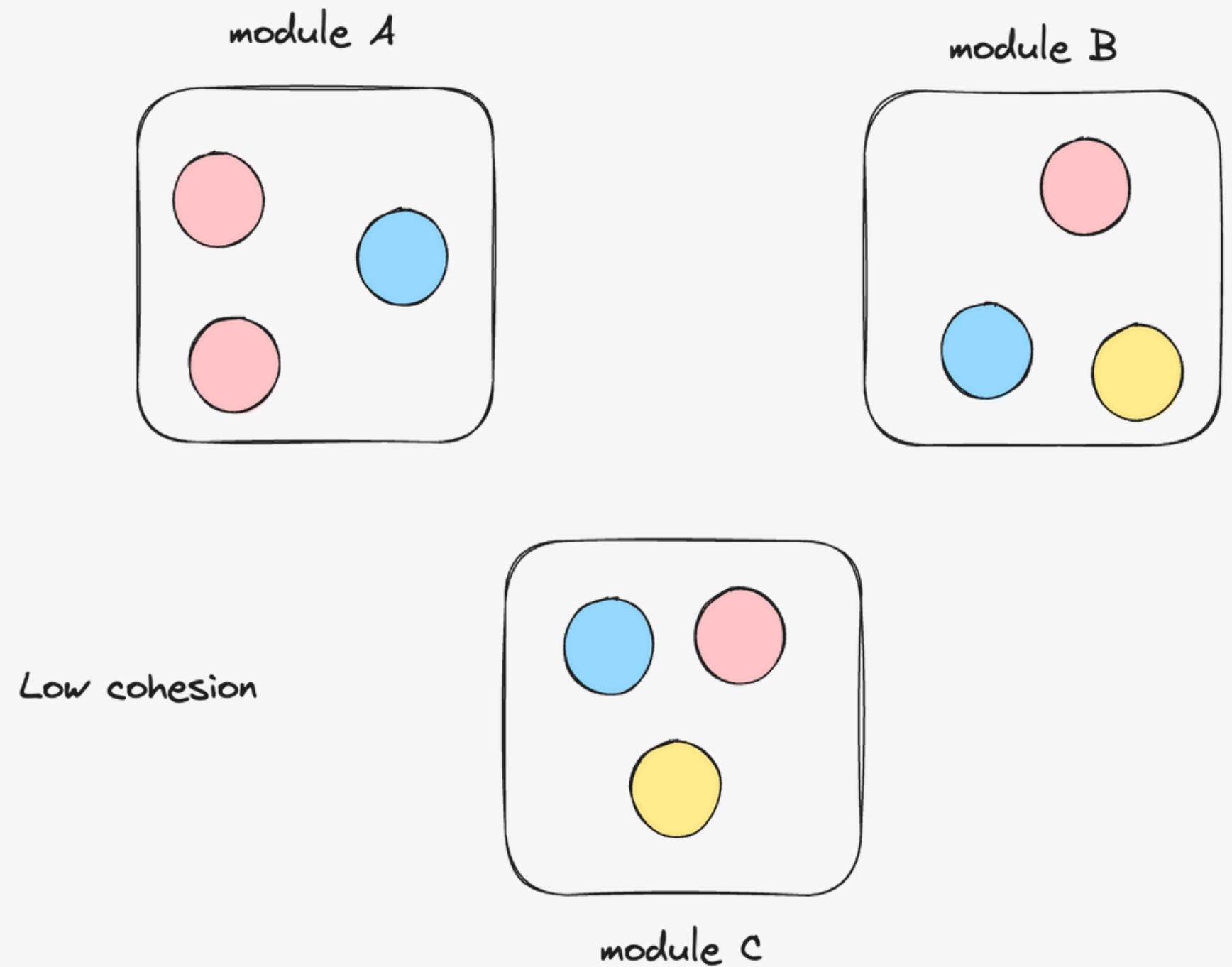
A **holistic approach** to software engineering that focuses on the **system as a whole** instead of a collection of isolated components

- It's not just FE vs BE, **it's both** of these combined

Cohesion & Coupling

Cohesion - how much your methods/variables within a module belong together

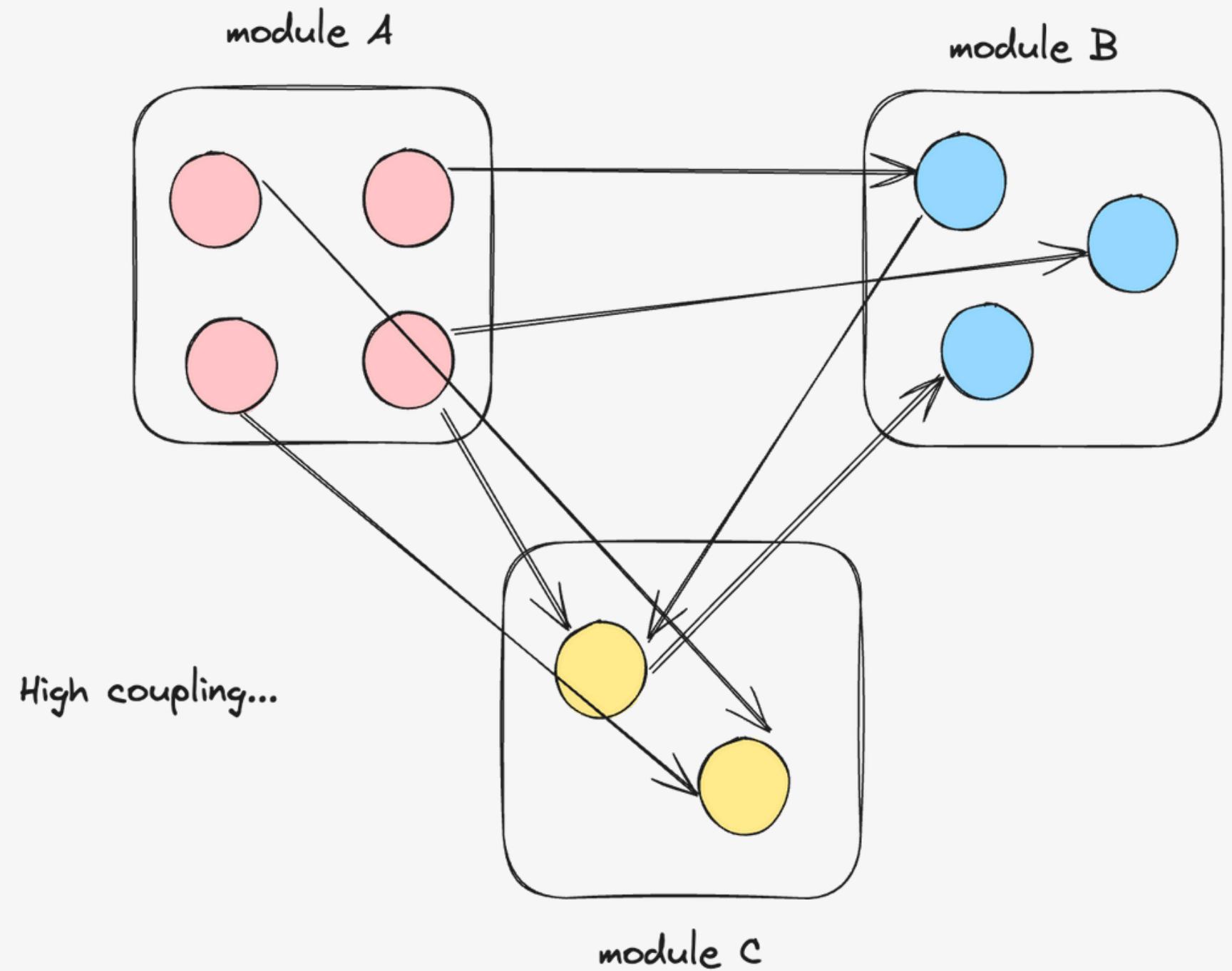
- ex: organization within modules



Cohesion & Coupling

Coupling - how **dependent** your software modules are to one another

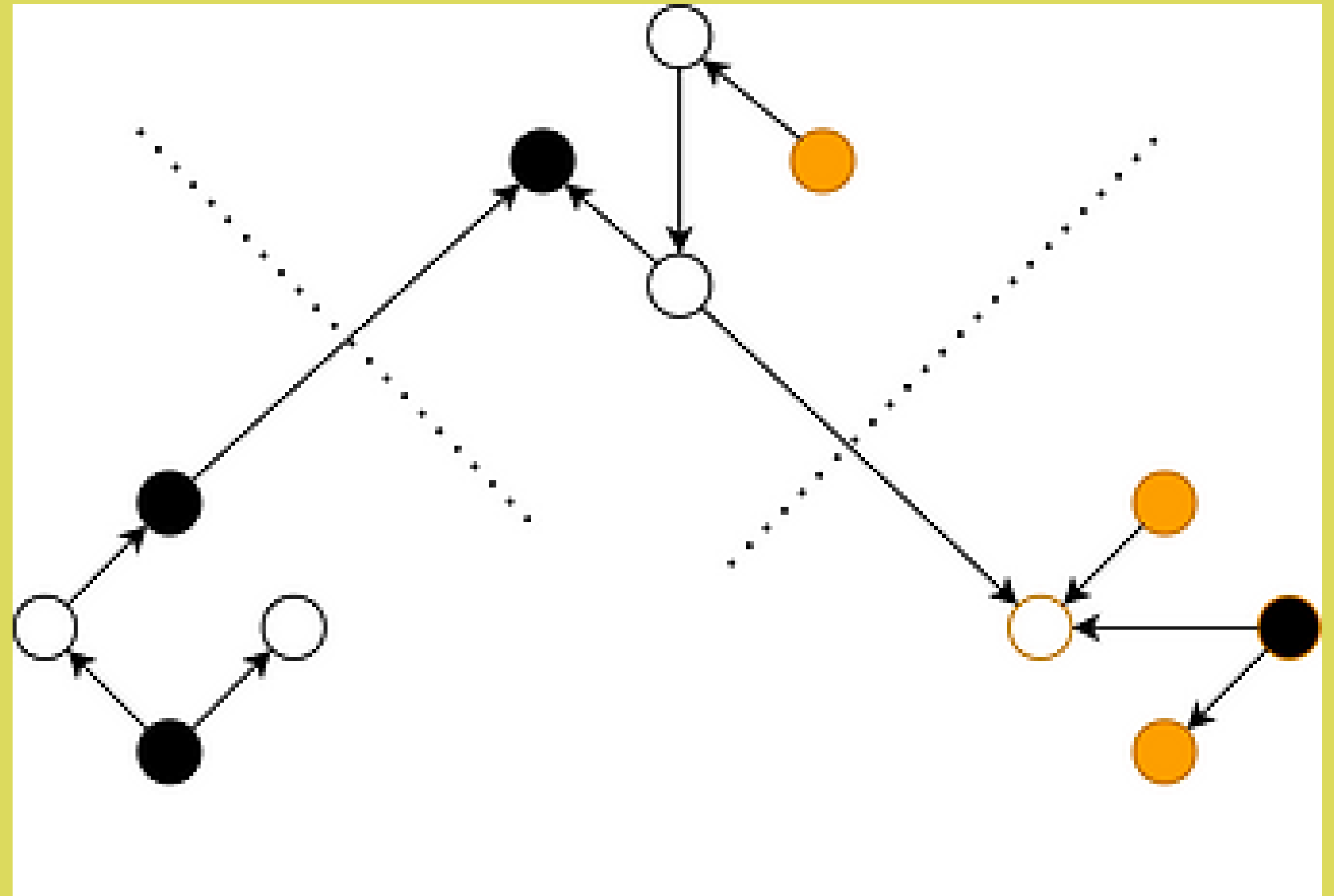
- ex: how many files would you need to change if you were to refactor something small?



Loose Coupling Low Cohesion

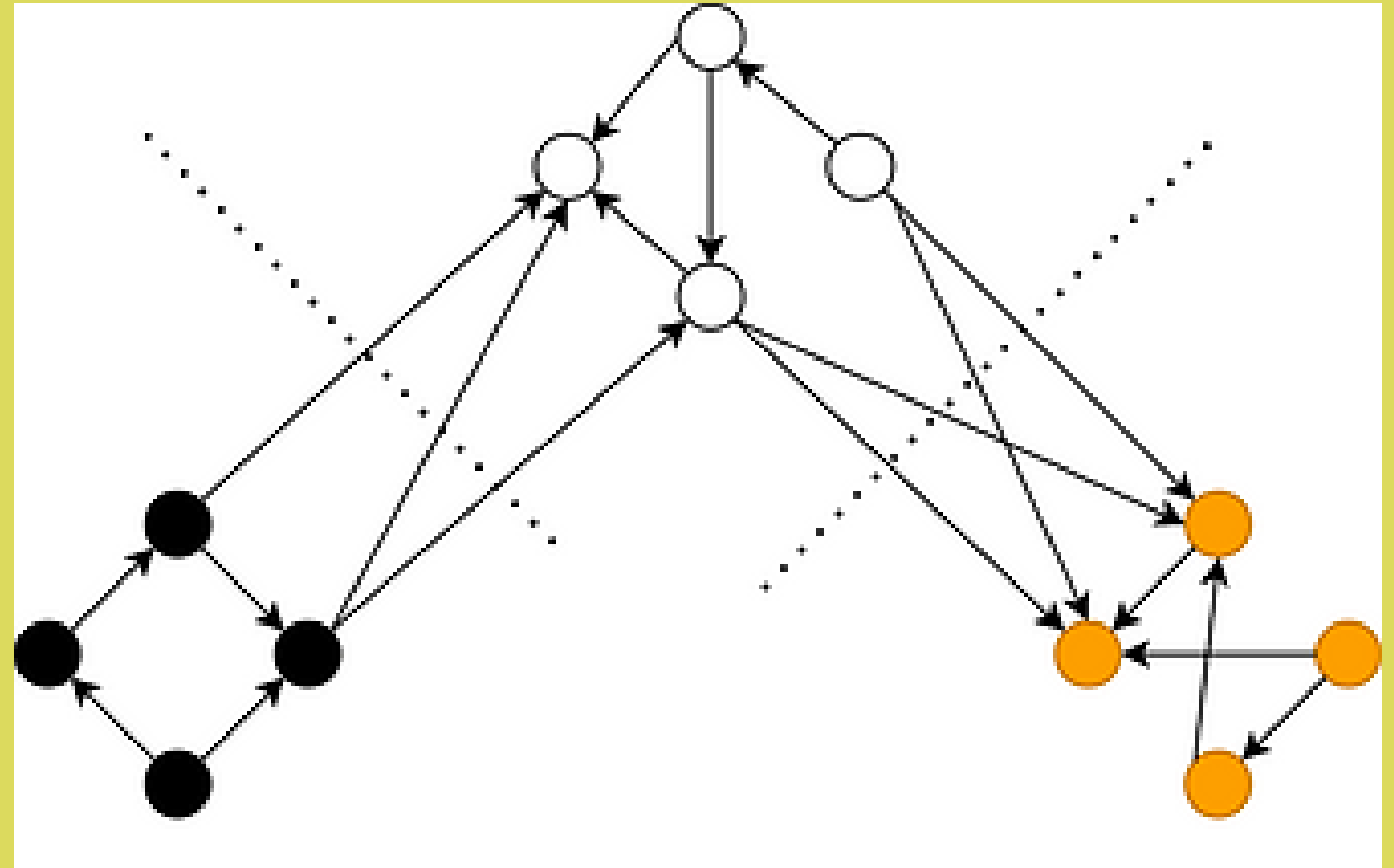
The Spaghetti code situation...

- Random things are placed in different modules
- The “I just want to make this work” approach



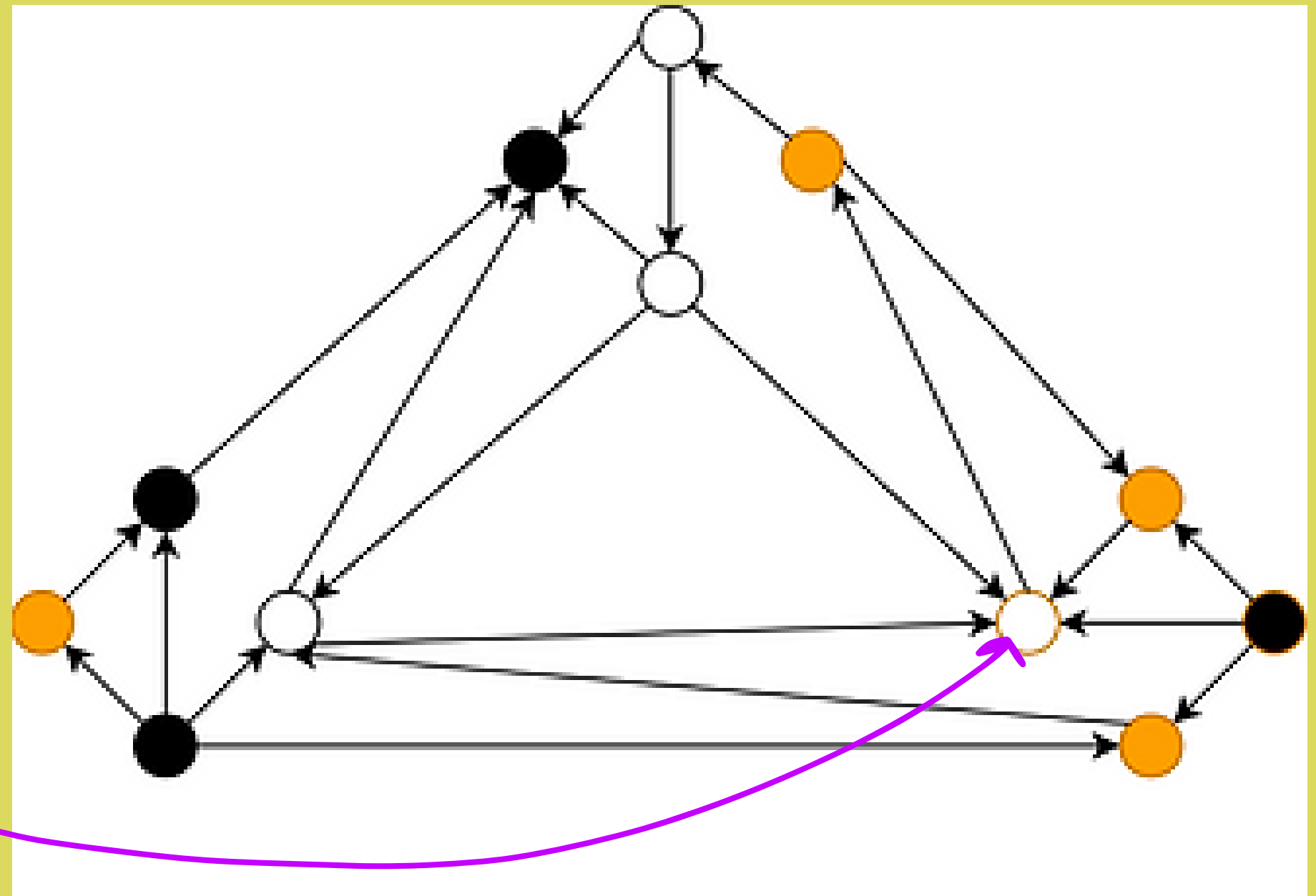
Tight Coupling High Cohesion

- Not enough reuse happening
- Modules have become monoliths that will make changes/updates/bugfixes difficult



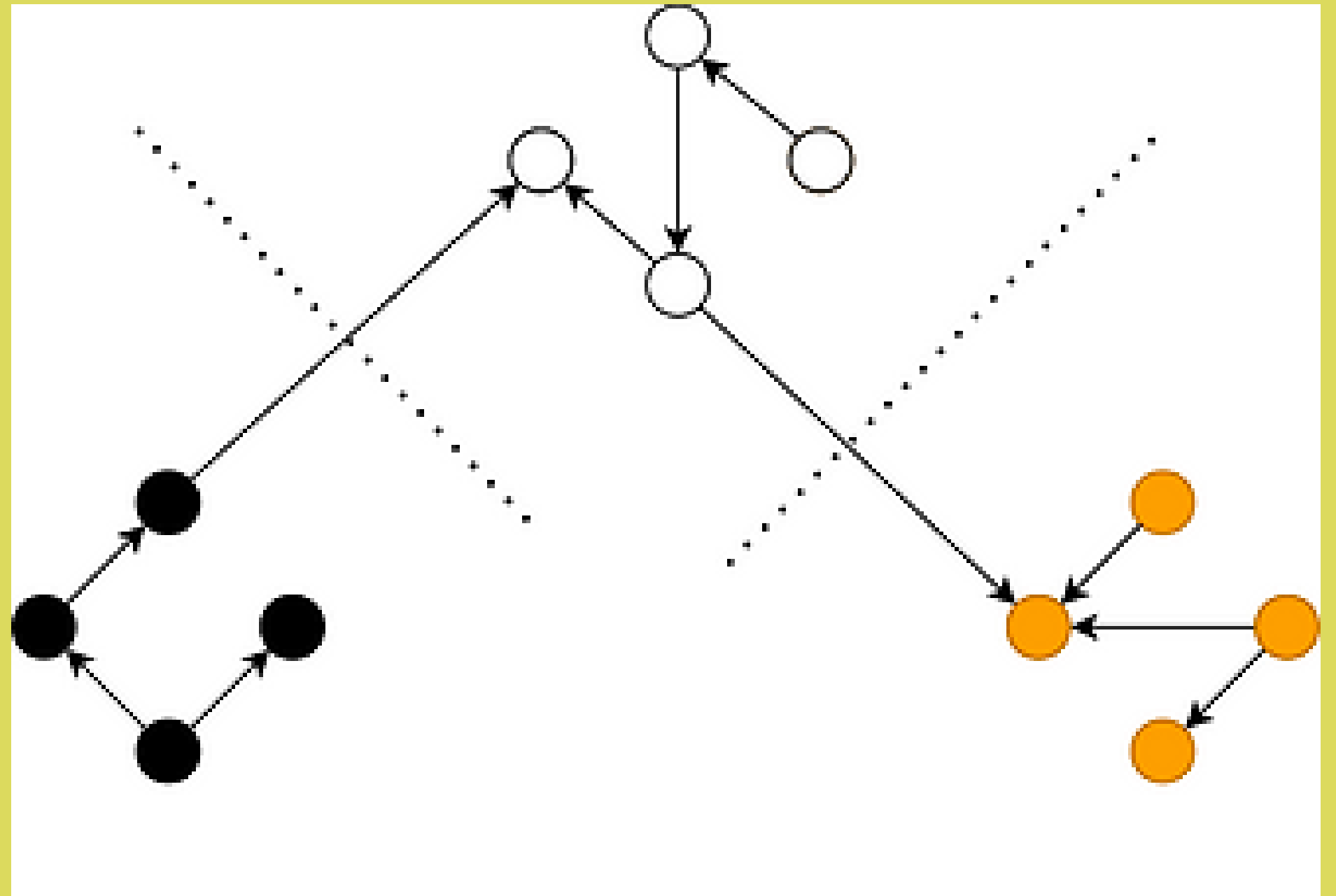
Tight Coupling Low Cohesion

- Creation of the ultimate all-knowing object



The Ideal: Loose coupling, High cohesion

- Clean code
- Easy to maintain
- Can add features/make changes much easier



A React context that has states for every single component in the entire component tree is an example of...

- a. Loose coupling, high cohesion
- b. Loose coupling, low cohesion
- c. Tight coupling, high cohesion
- d. Tight coupling, low cohesion



bit.ly/3ELO07B

Architecture vs Design Patterns

Architecture is the scaffolding that everything else is built on top of

Design patterns focus on how to structure the internal workings to solve problems

Architecture

**“...getting something to work--once--just isn't hard.
Getting it right is another matter entirely.
Getting software right is hard.”**

-Clean Architecture by Robert C. Martin

Cool...but why?

Foundational knowledge so you can learn pretty much any framework

Because it makes you more productive in the long run

We'll always be pressured to move faster to go to market ("just make it work mentality")

Don't sweat the small stuff that can be easily changed (ex: spending toooooo much time on a variable name)

Focus on the stuff that you need to get right from the beginning...architecture

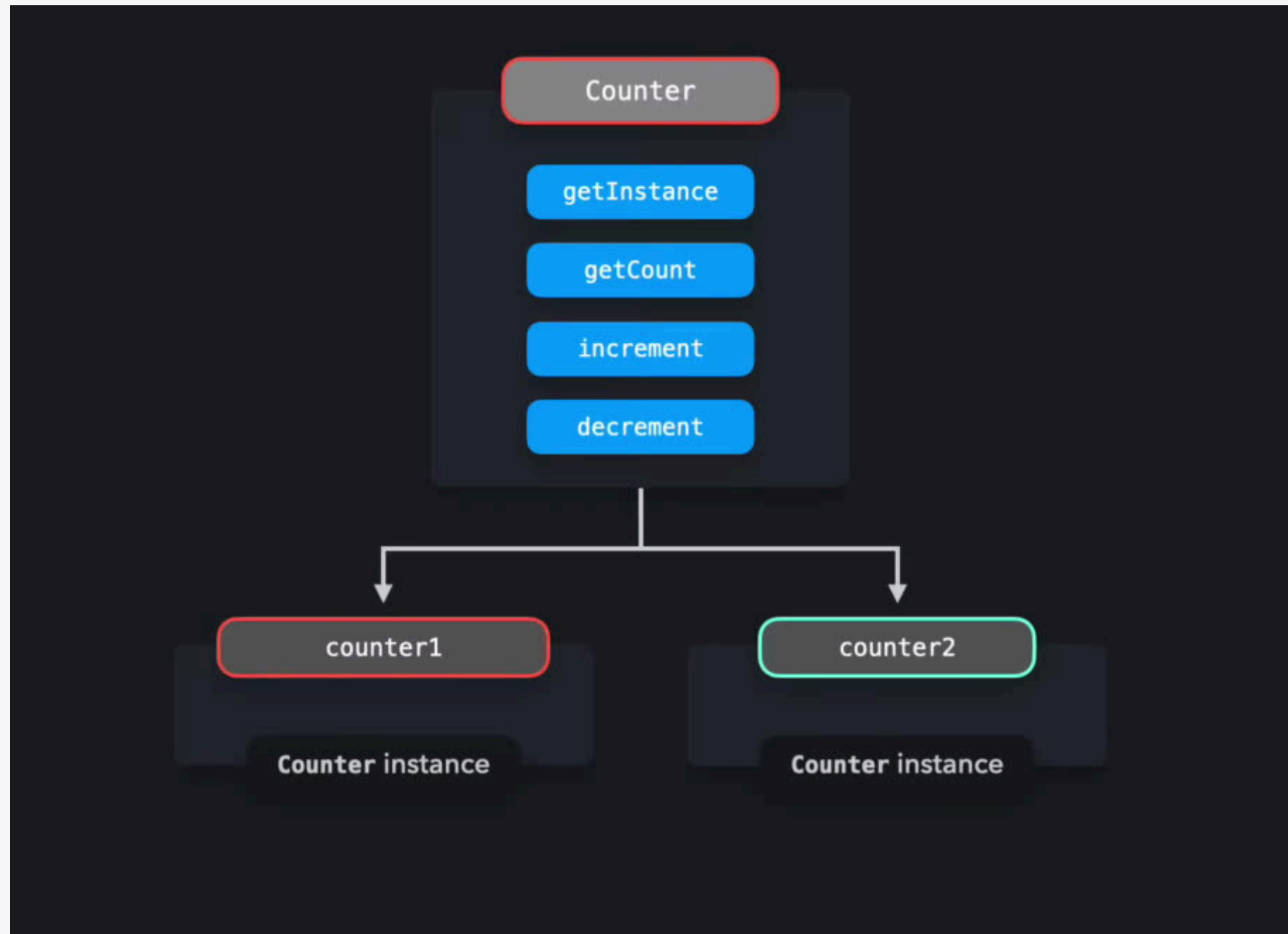
- For scalability and performance

Common patterns in FSSE

- **Singleton**
- **Observer**
- **Decorator**
- **Strategy**

Common patterns in FSSE

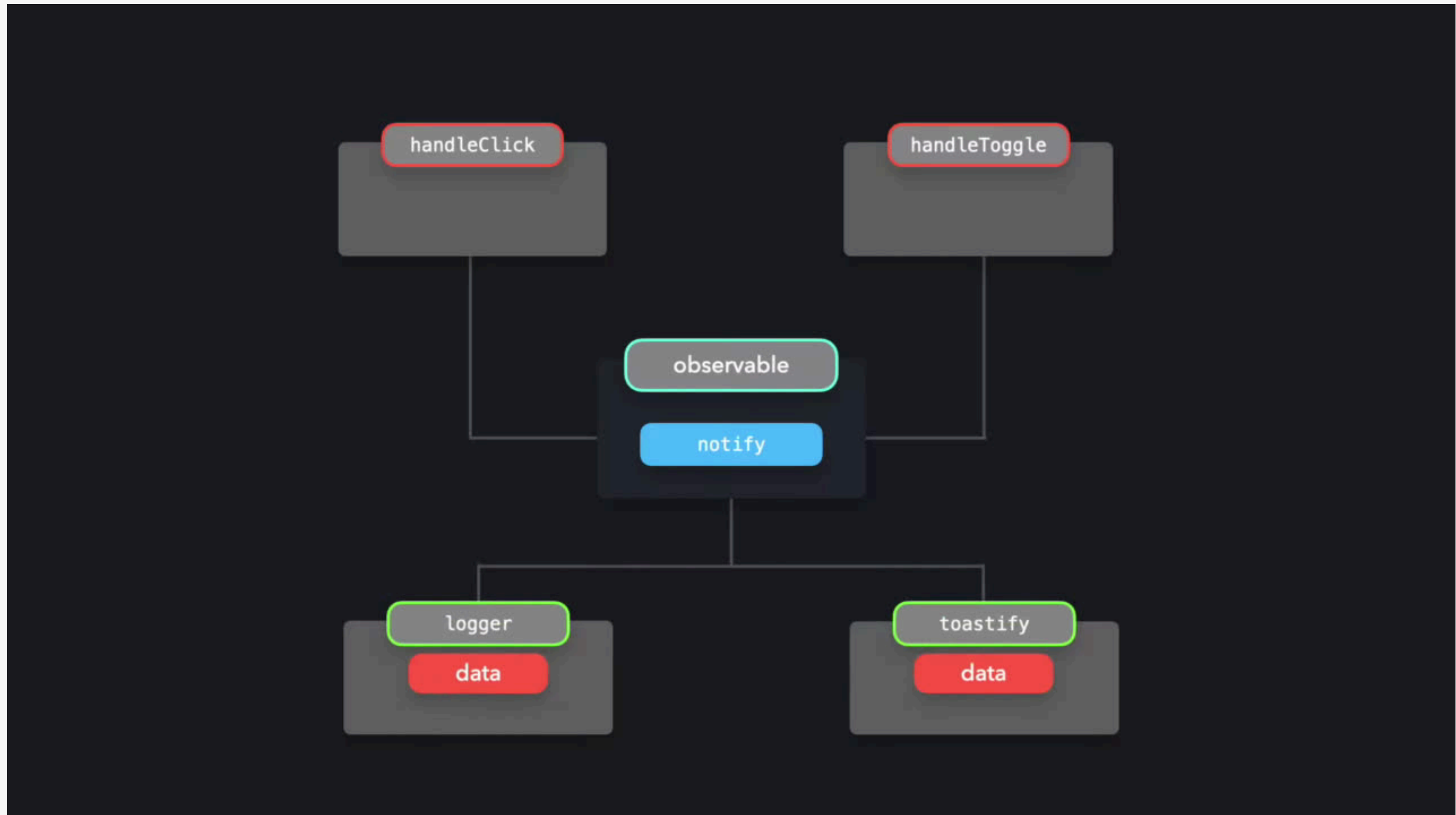
Singleton - you reference a single instance of a class throughout your app



Common patterns in FSSE

Observer - when an object subscribes as an observer to another object (observable)

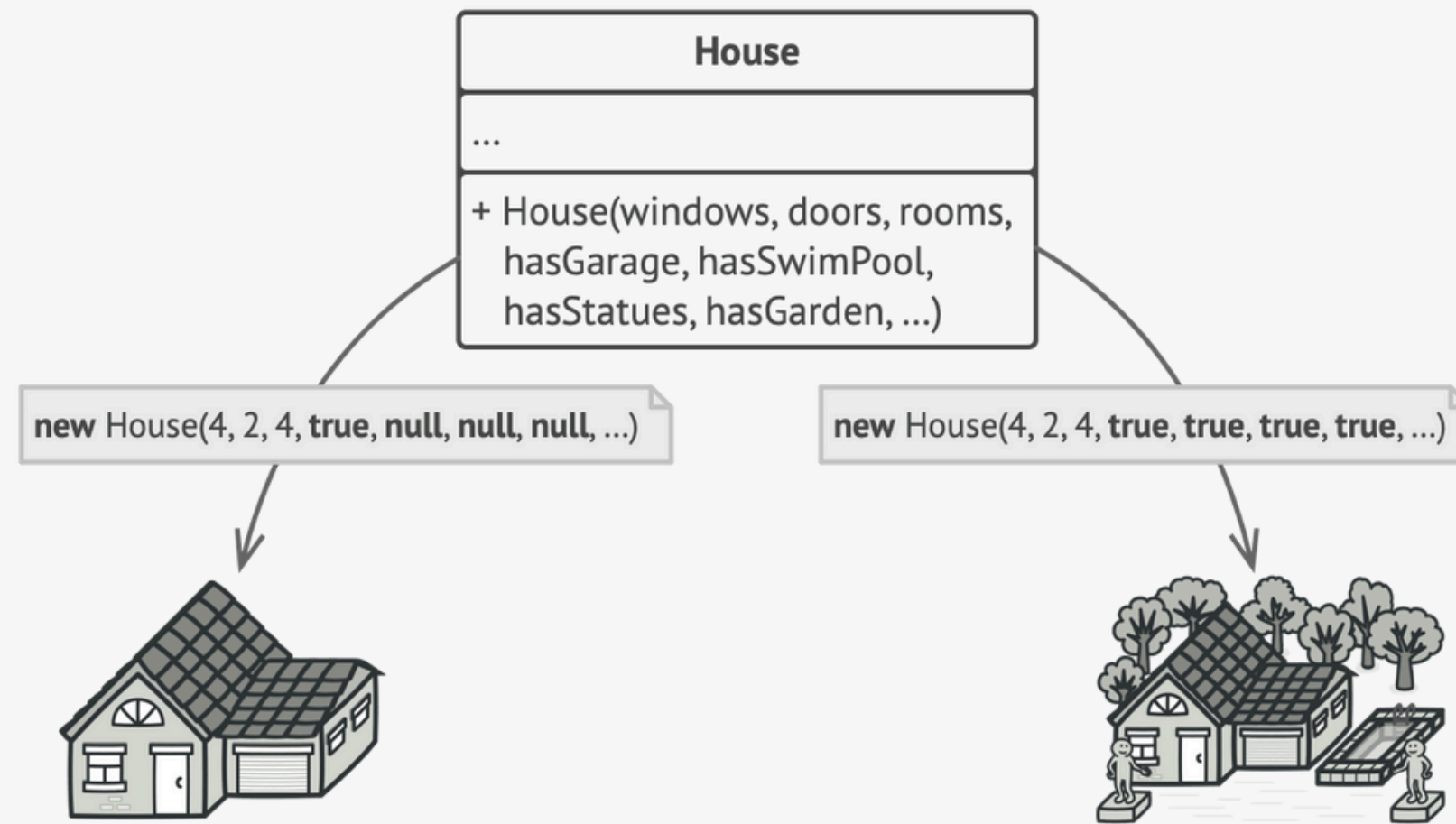
- When something happens to the observable, it will notify all observers



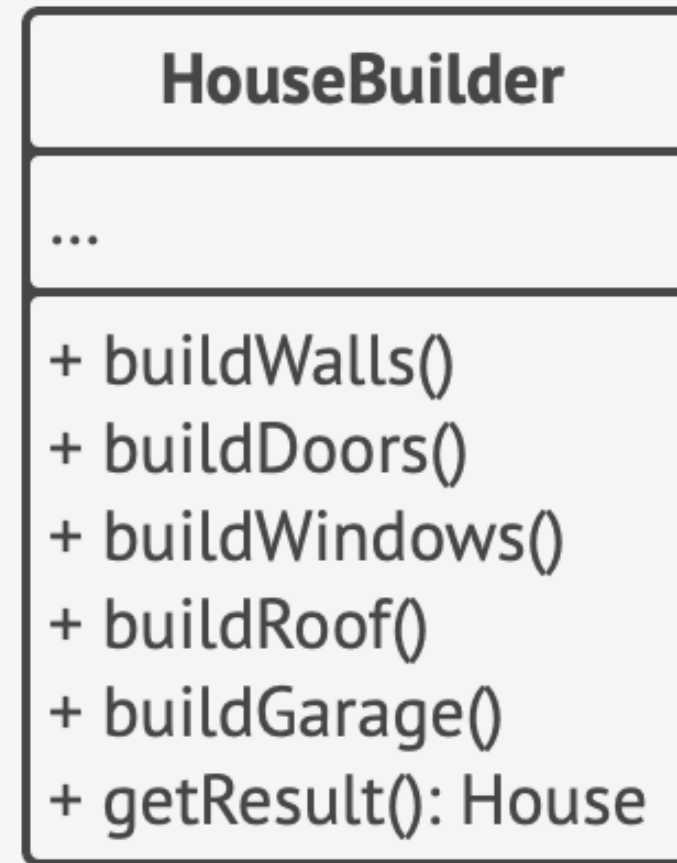
Common patterns in FSSE

Builder - separates object creation from adding details about the object

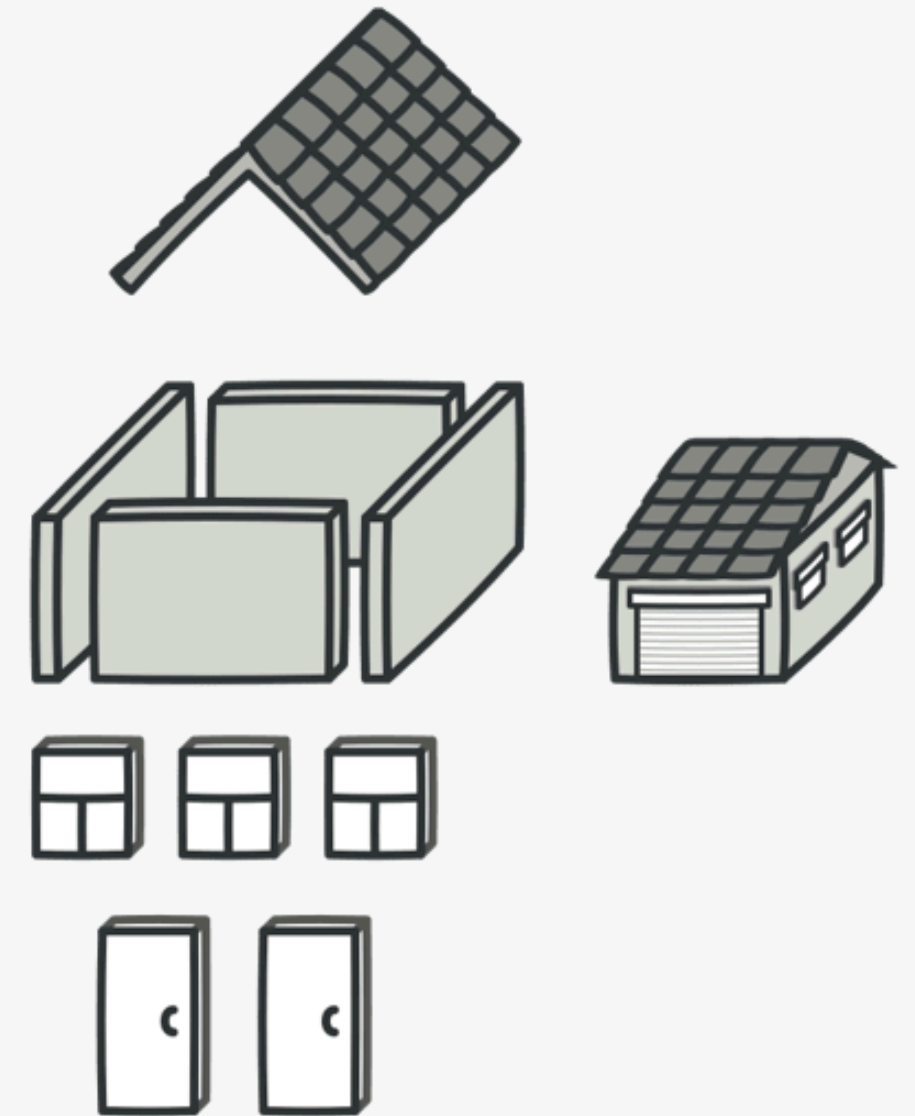
- example



before



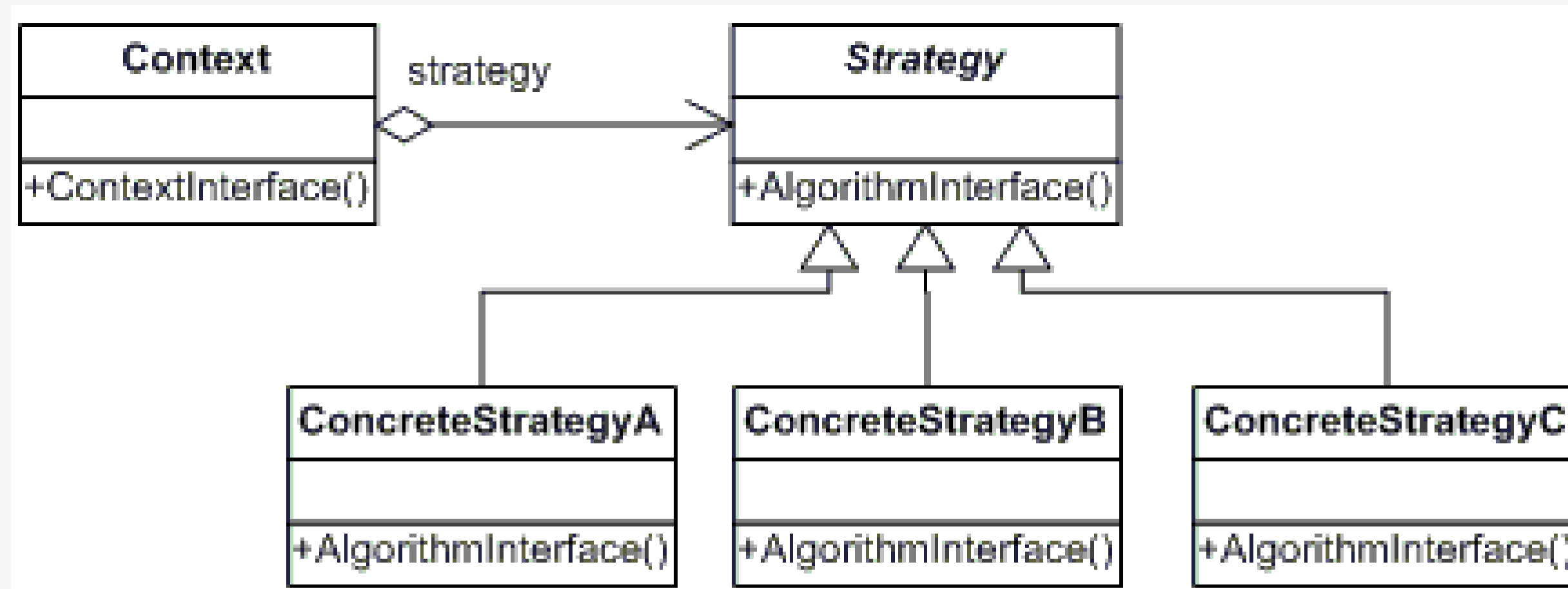
after



Common patterns in FSSE

Strategy - when you need to use the same algorithm with minor variations (eliminates complex conditionals)

- example



Application Architecture

Patterns: MV*

- **MVC**
- **MVP**
- **MVVM**

Application Architecture

Patterns: MV*

Common Terms

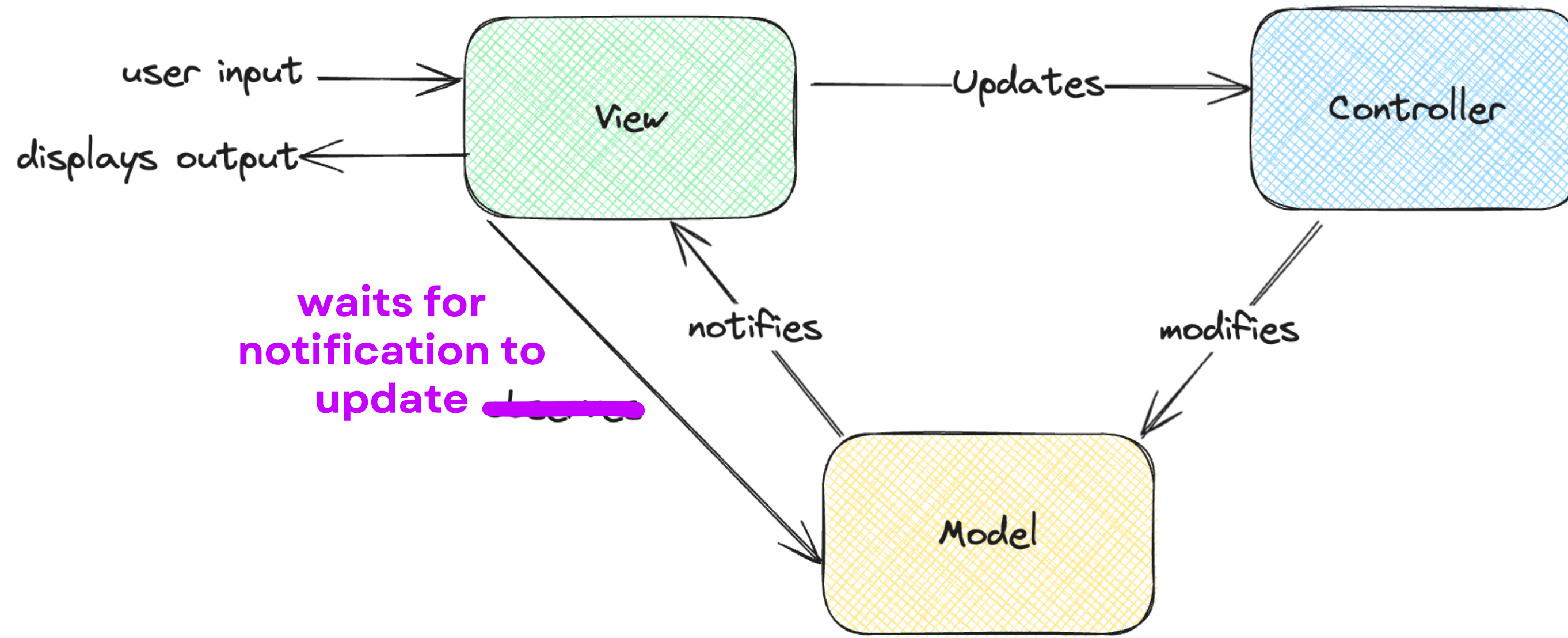
Model - manages data for an app

View - visual representation of the model

Application Architecture Patterns: MV*

Model View Controller (MVC)

Controllers - in charge of updating the model



Which pattern best describes the relationship between a view and model in MVC?

- a. Singleton
- b. Observer
- c. Strategy
- d. Decorator

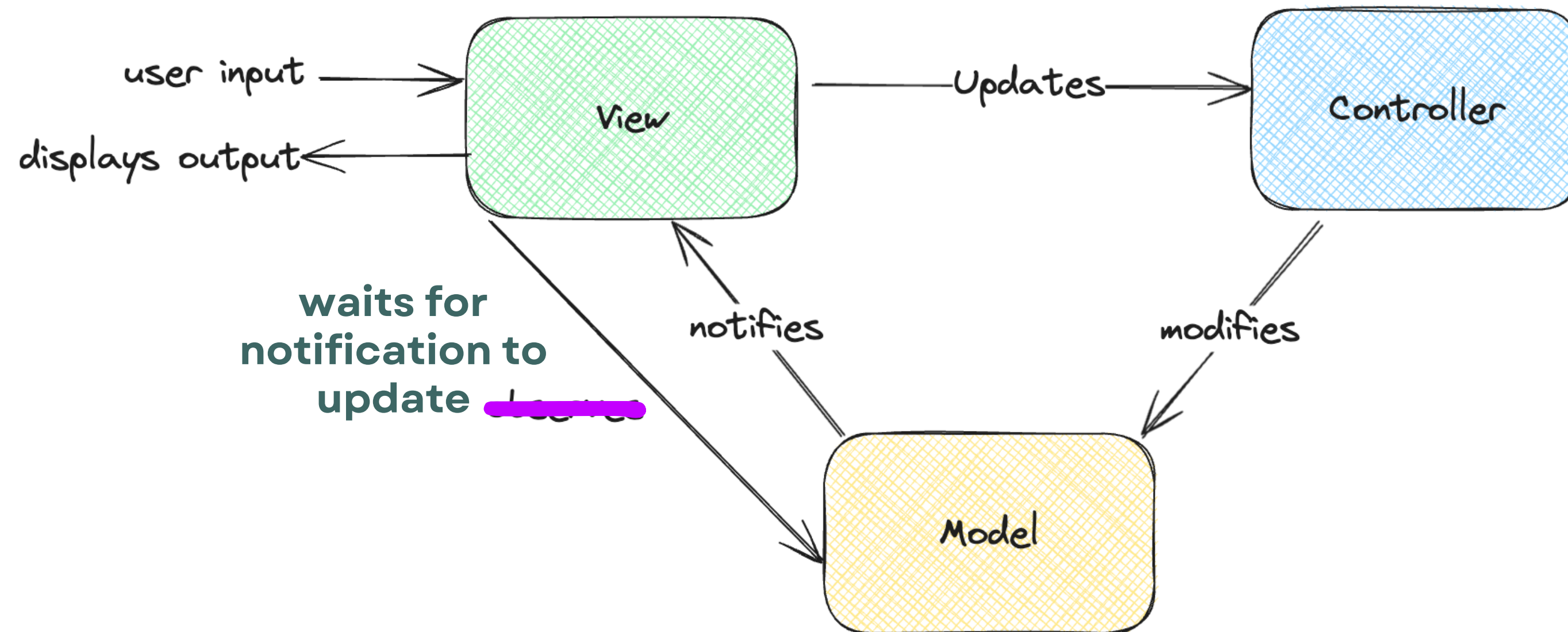


bit.ly/4dbUepr

Application Architecture Patterns: MV*

Model View Controller (MVC)

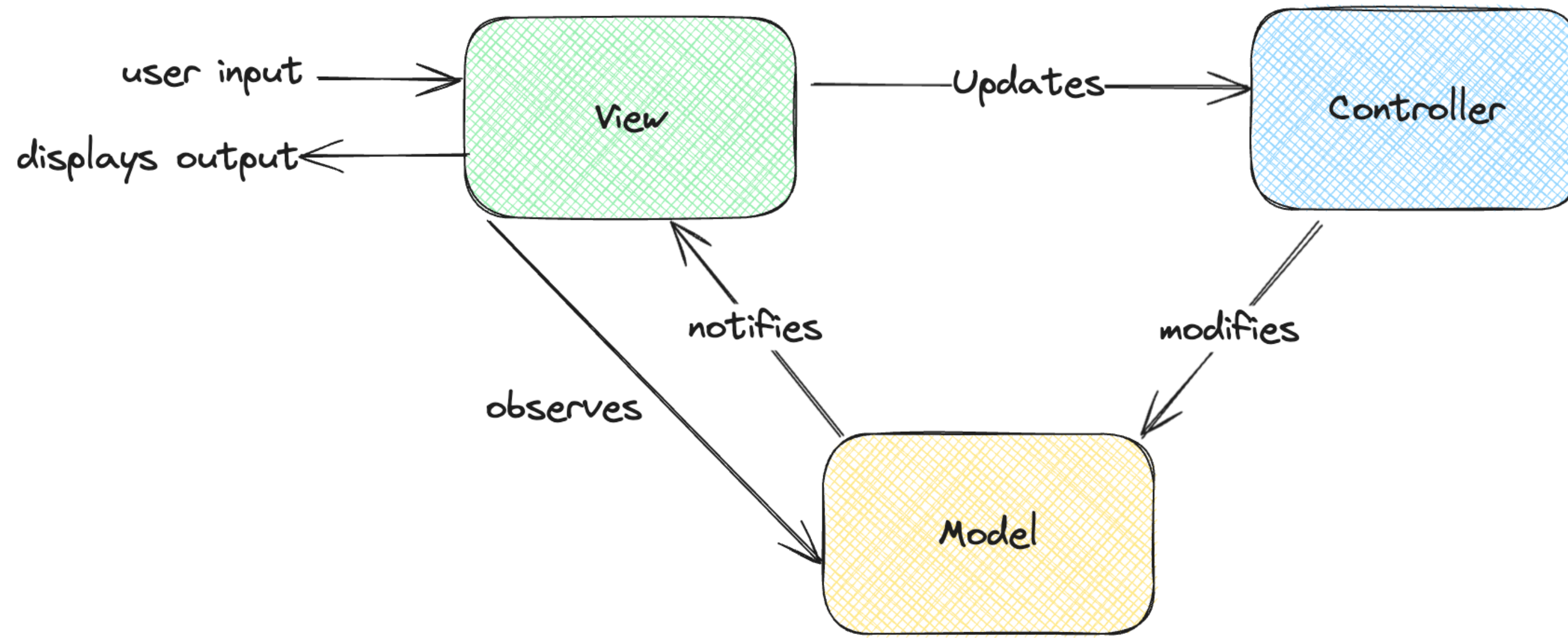
Controllers - in charge of updating the model



Application Architecture Patterns: MV*

Model View Controller (MVC)

Controllers - in charge of updating the model



Which pattern best describes the relationship between a view and controller in MVC?

- a. Singleton
- b. Observer
- c. Strategy
- d. Decorator

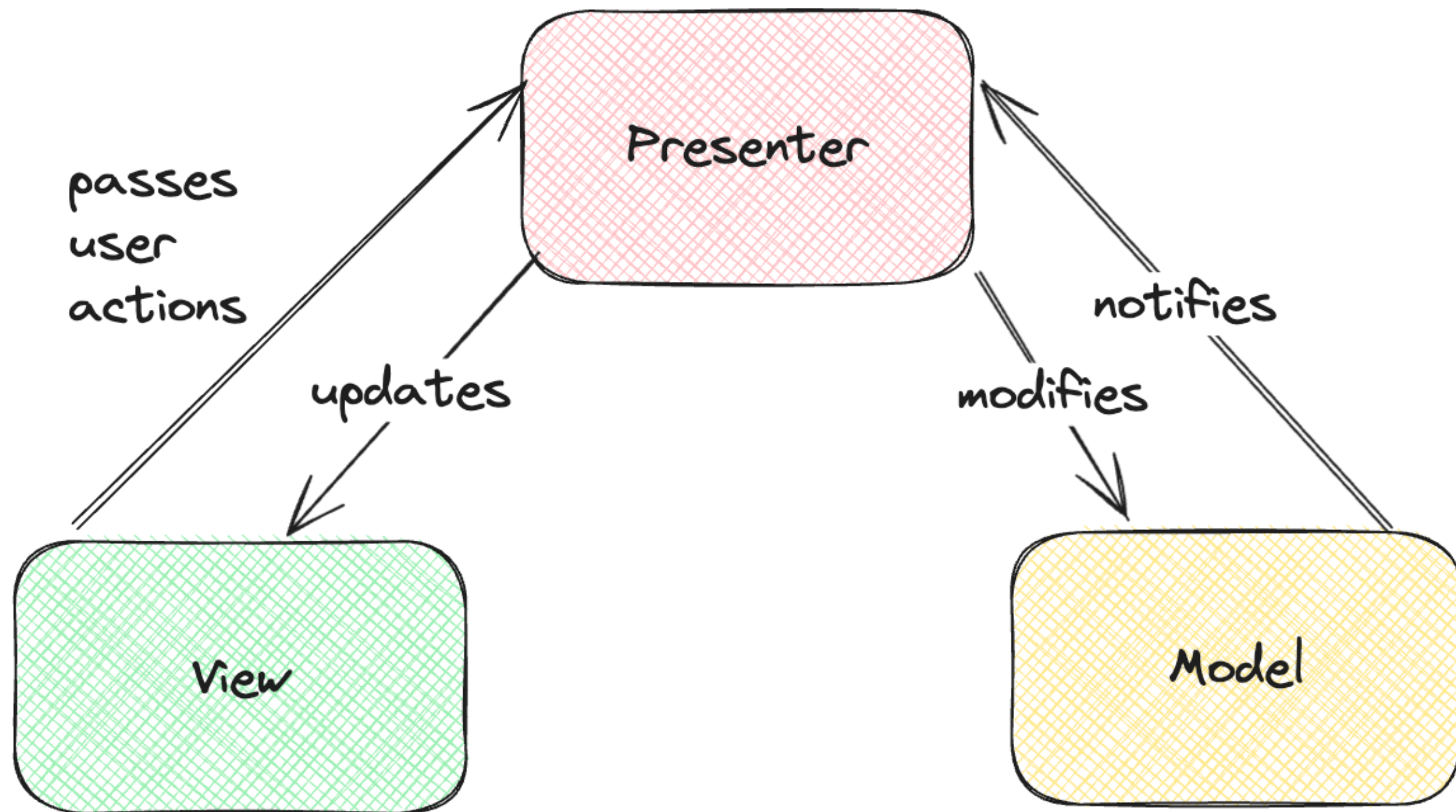


bit.ly/44df995

Application Architecture Patterns: MV*

Model View Presenter

Presenter - handles modifying the model and the view

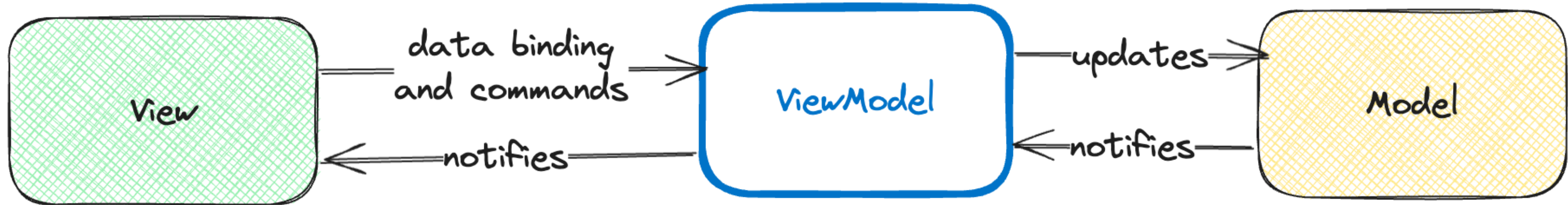


Application Architecture Patterns: MV*

Model View ViewModel

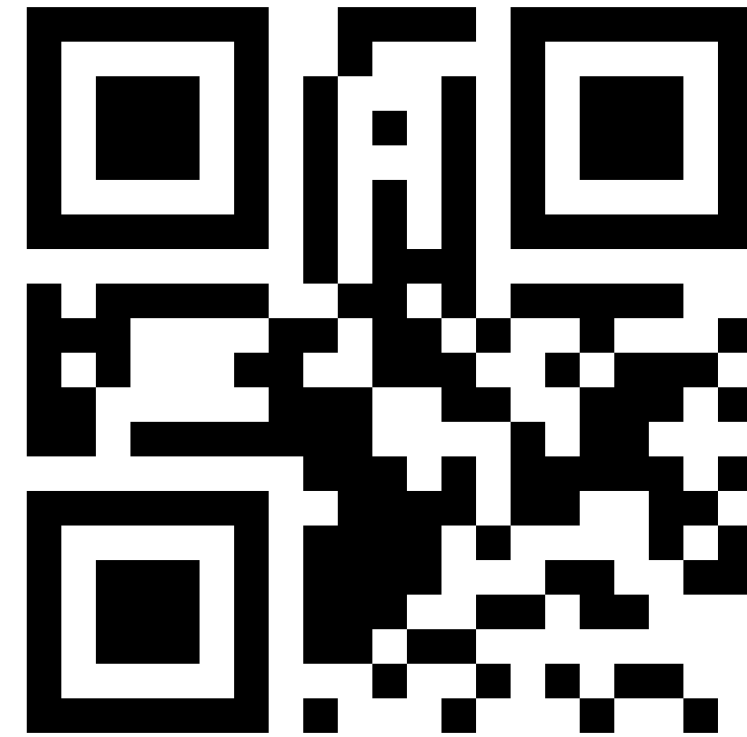
ViewModel data converter with methods exposed to view

View is active



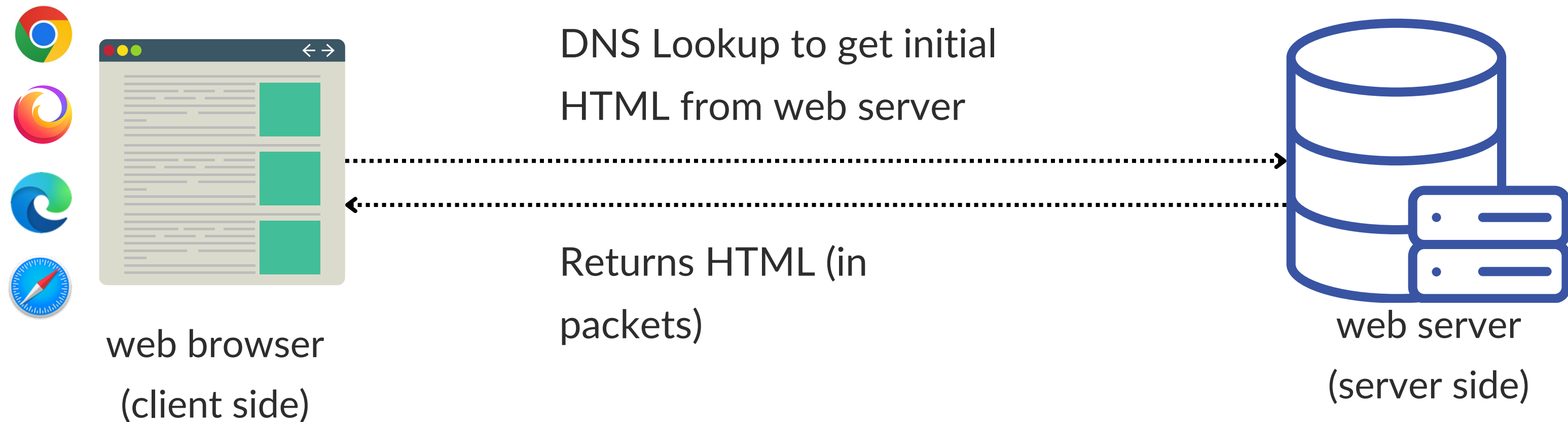
True/False: React is an MVC framework

- a. True
- b. False

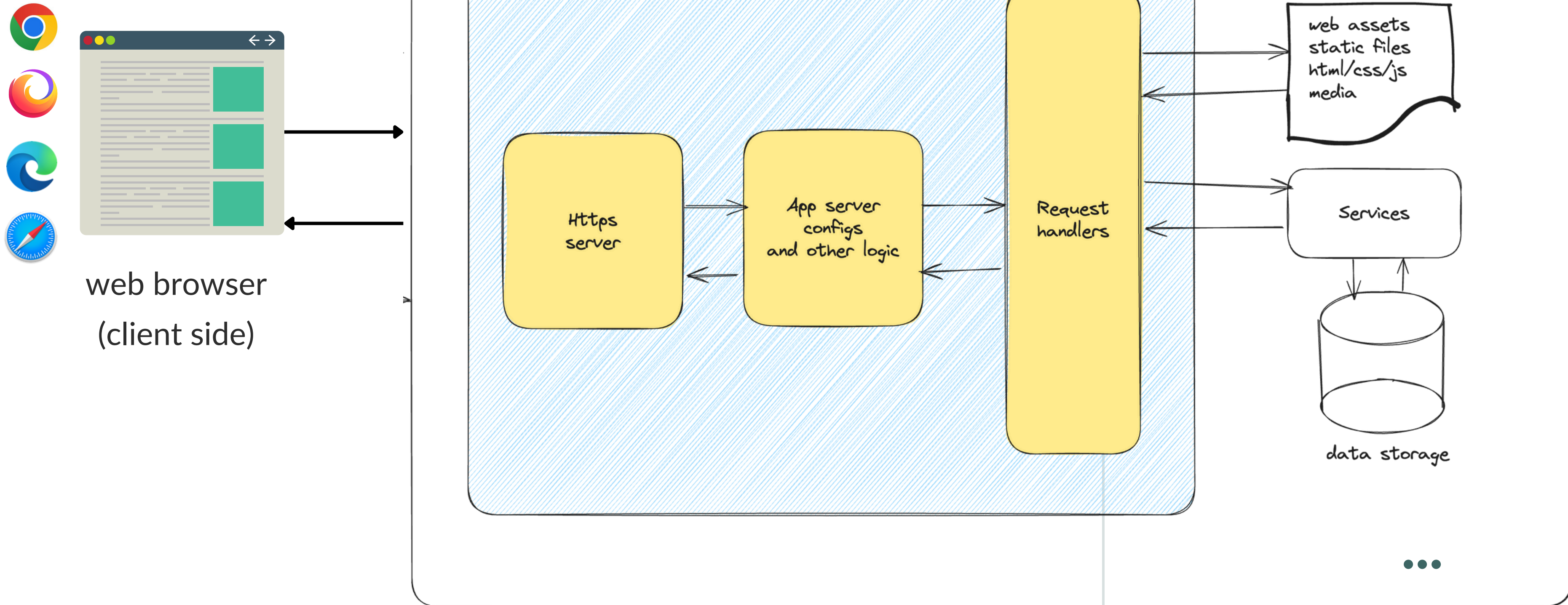


bit.ly/4jLBoHq

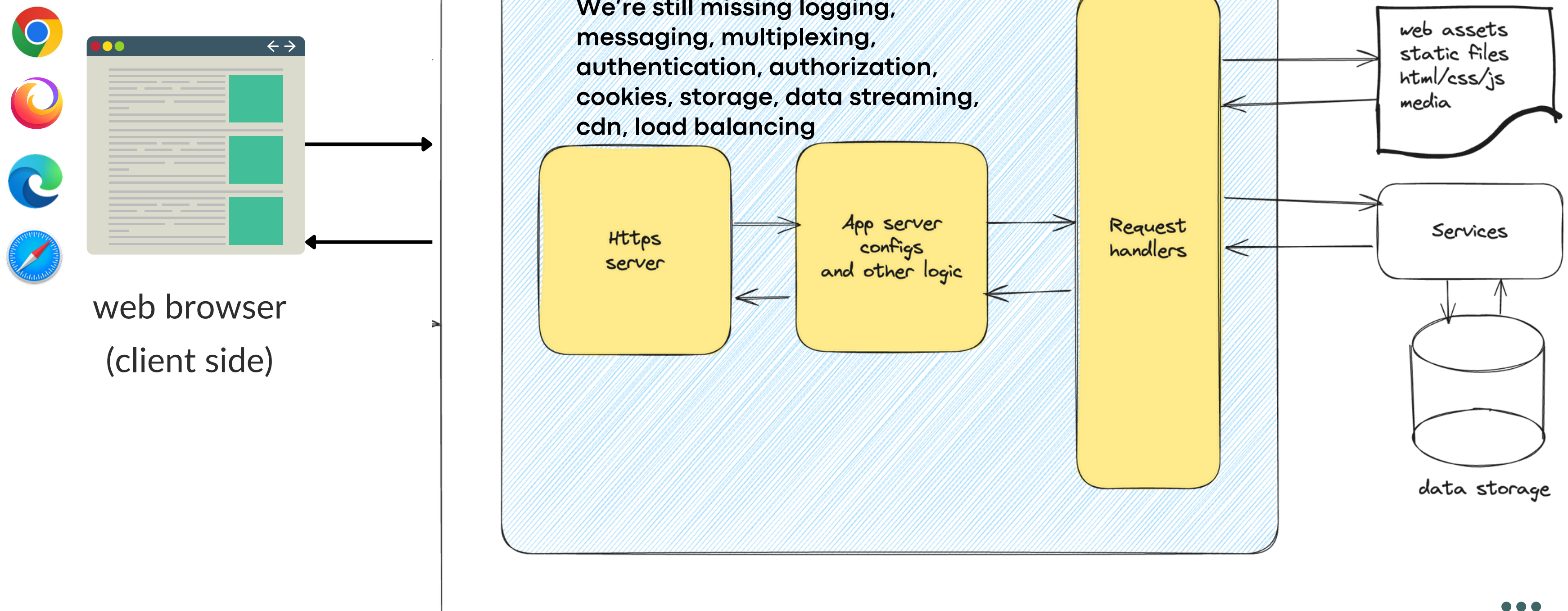
The web app that we all know...



The web app that we all know...



The web app that we all know...



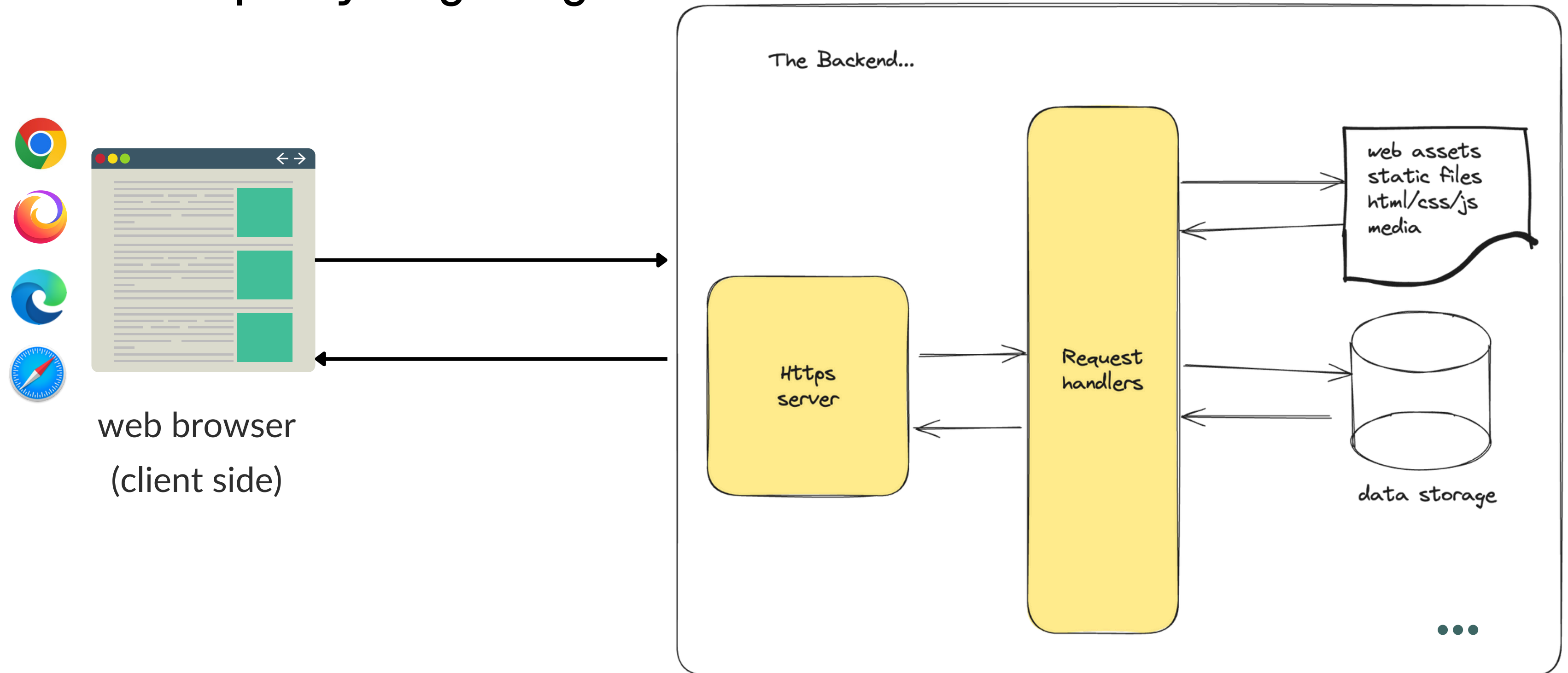
Architecture Patterns for the BE

- SPA (Thin server architecture)
- Microservices
- Serverless
- Layered
- Event based/driven* (we'll discuss next week with sockets)

Patterns for the BE

SPA (Thin server architecture)

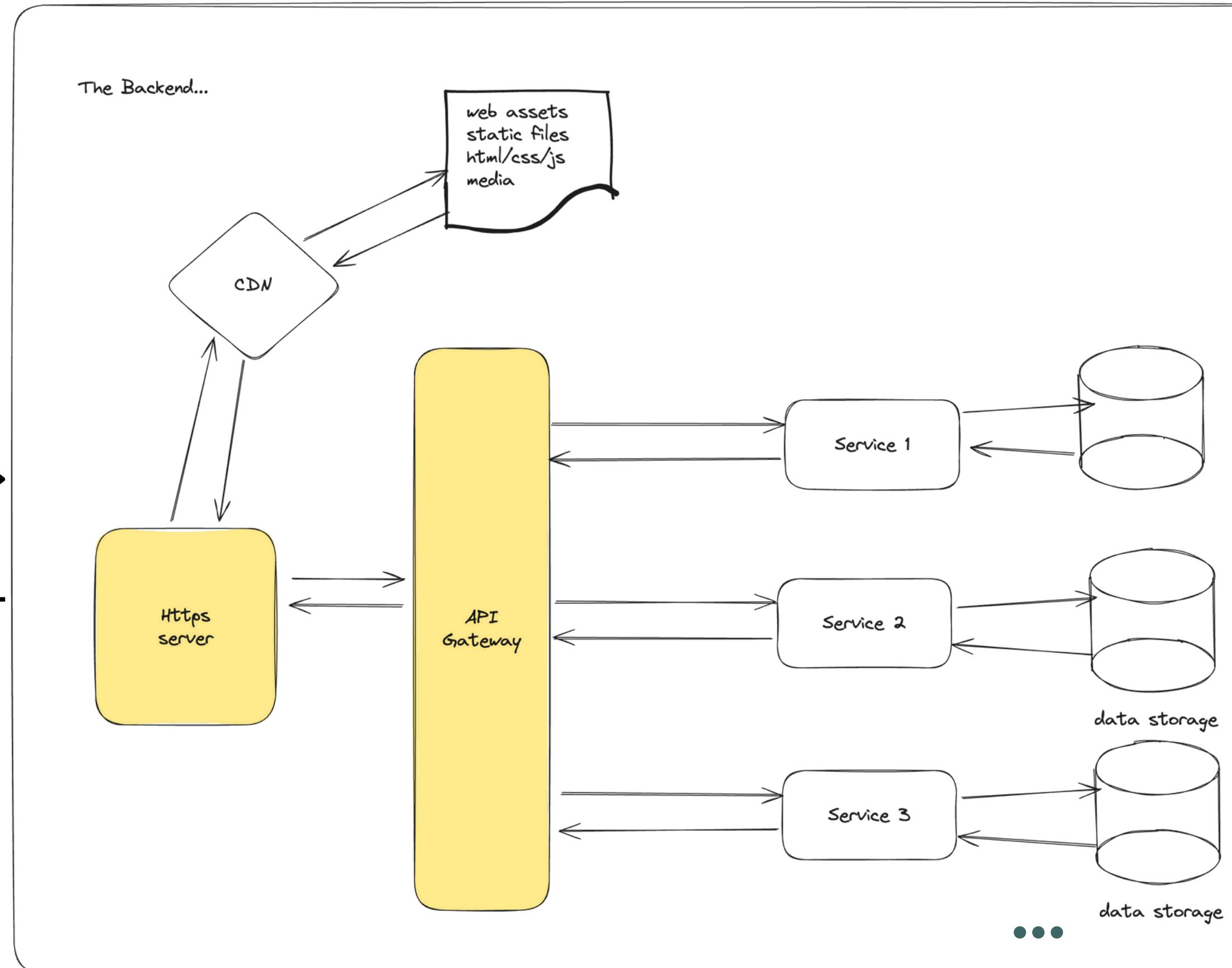
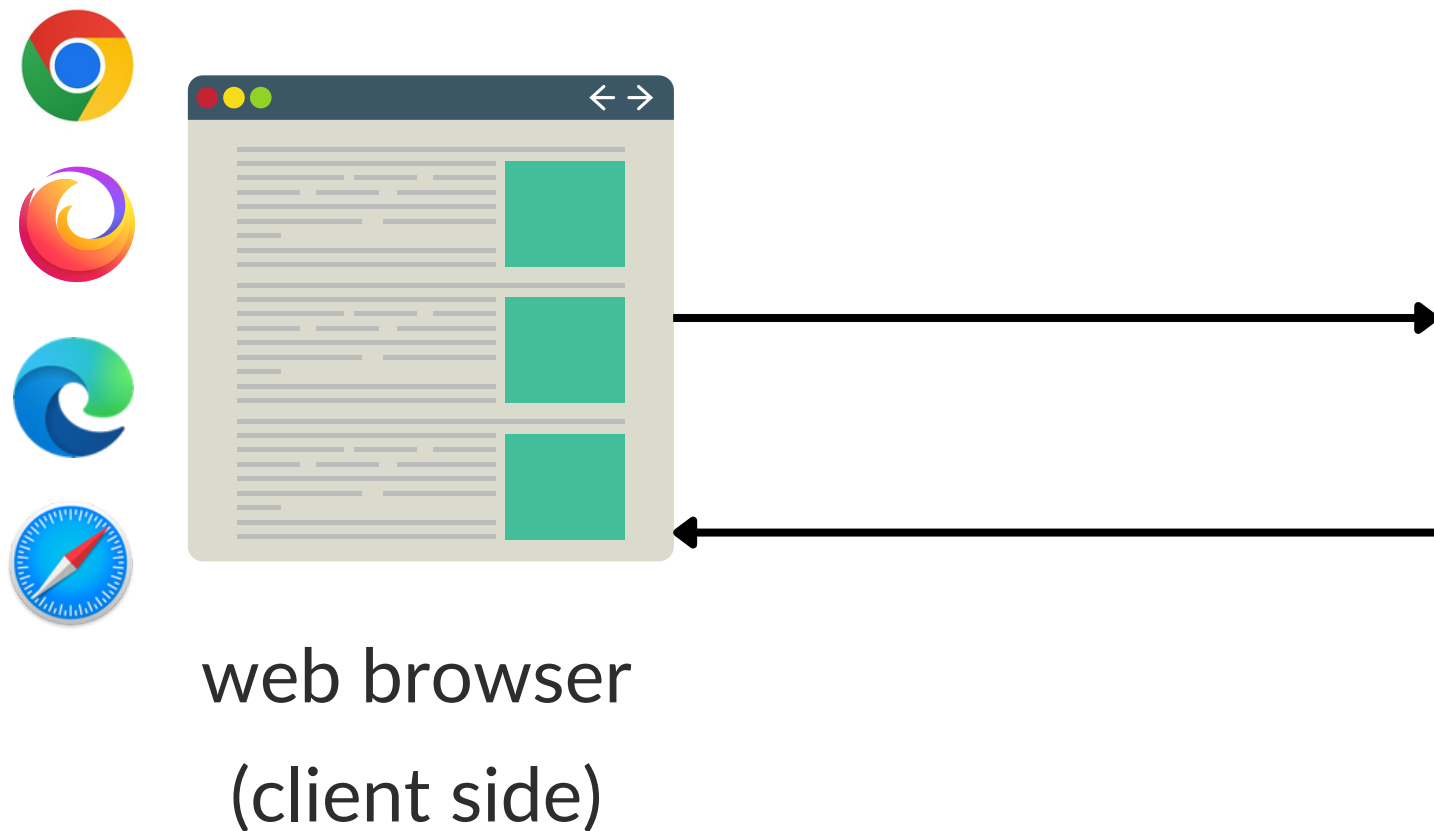
- client performs a lot of the business logic
- server is purely for getting data



Patterns for the BE

Microservices

- When you break up a large app into smaller independent pieces
- Each service has one purpose



Architecture Patterns for the BE

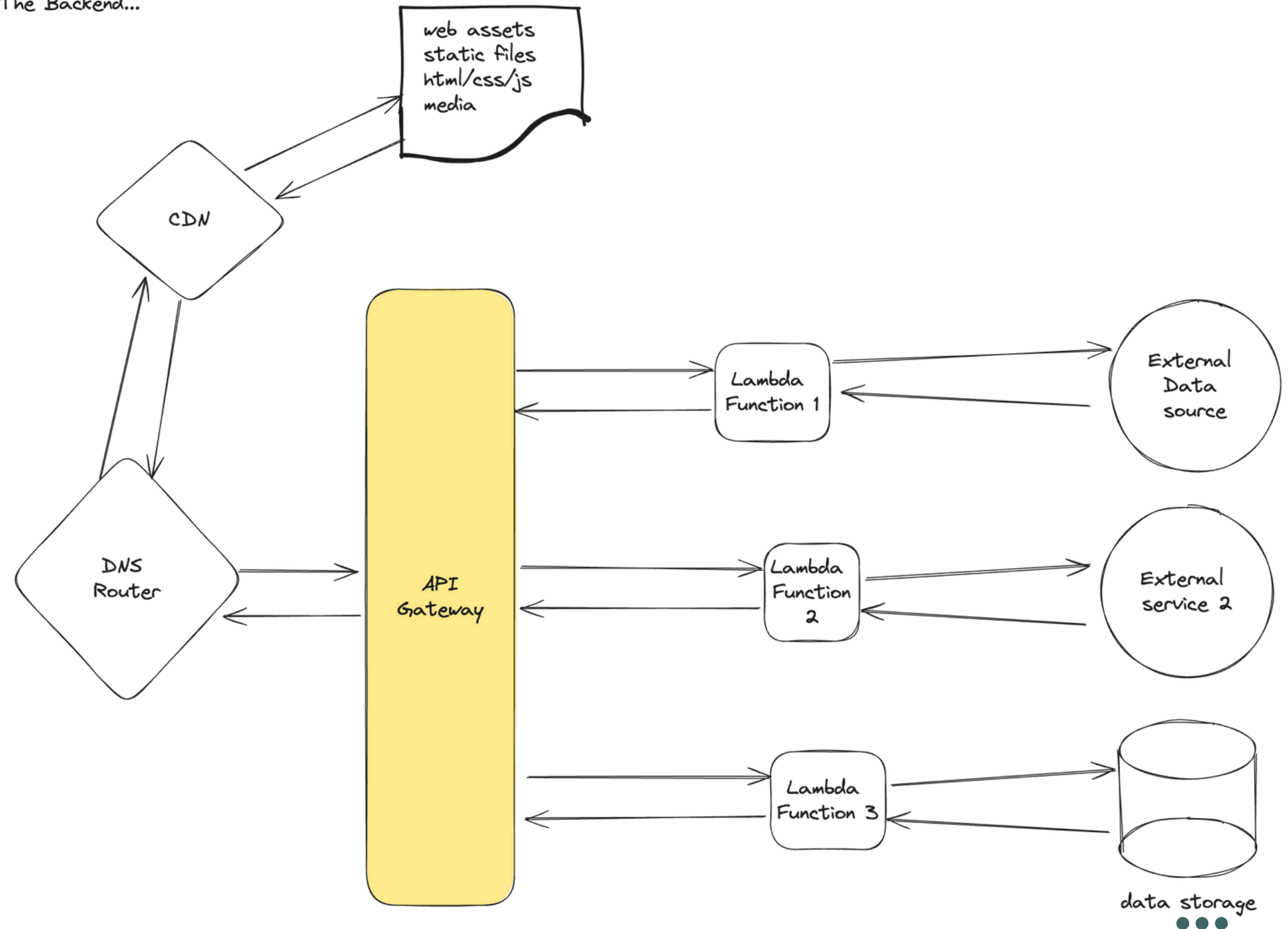
Serverless

- Infra managed by cloud service provider (ex: AWS) and lambdas are used to manage resources



web browser
(client side)

The Backend...



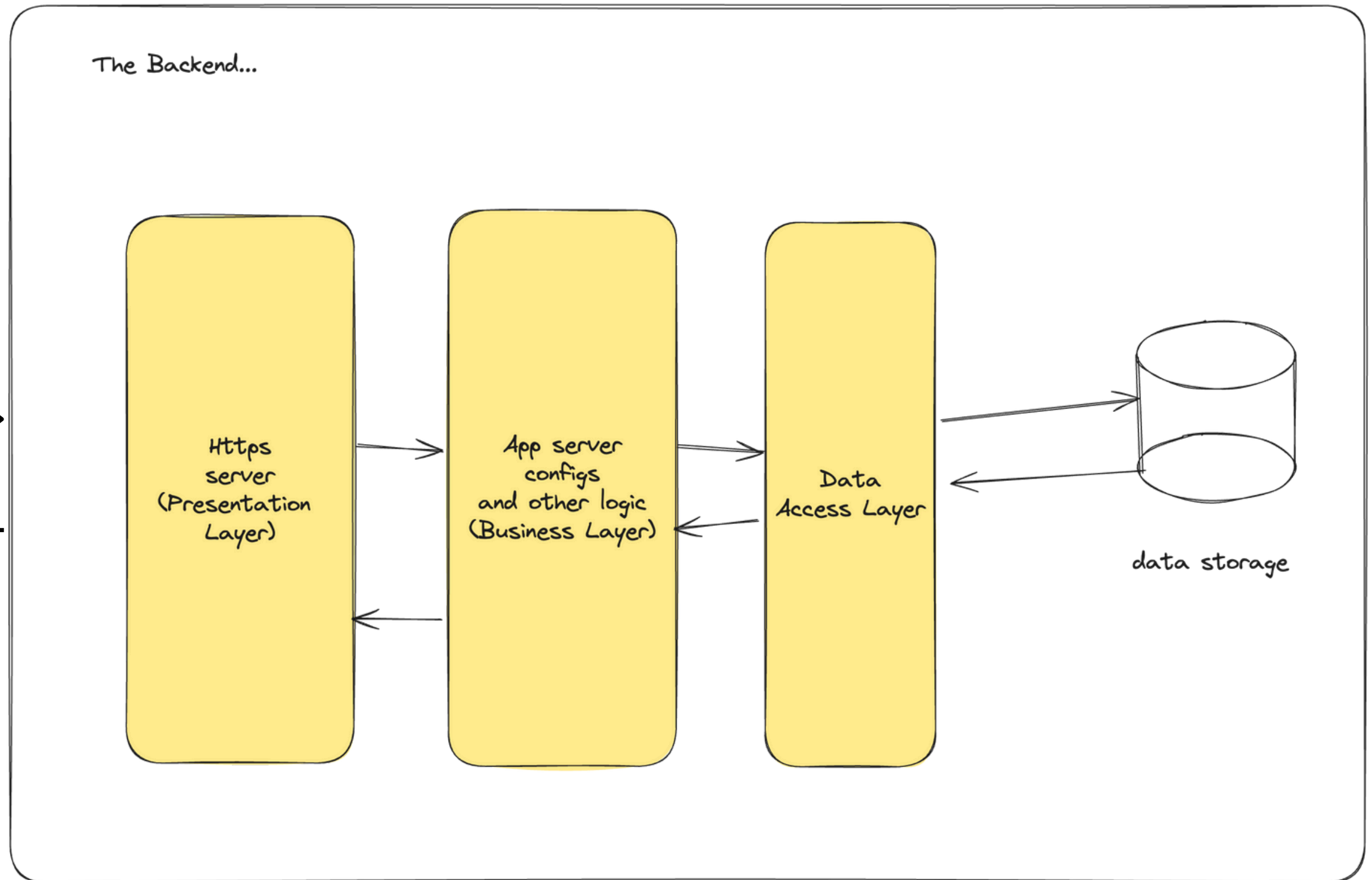
Architecture Patterns for the BE

Layered (monolithic)

- Entire app is one unit with multiple tiers



web browser
(client side)



Which BE architecture would lead to a lot of downtime when updating/performing maintenance?

- a. SPA (Thin server architecture)
- b. Microservices
- c. Serverless
- d. Layered



bit.ly/4jpL0YI

Frameworks for the BE

- Django (Python)
- Express (JS)
- Spring boot (Java)
- Ruby on Rails (Ruby)